S3 Static Website Hosting, Versioning, Windows EC2 Web Hosting and S3 Redirection

AWS Practical Experiment

1. Objective

The objective of this experiment is to configure Amazon S3 for static website hosting, demonstrate S3 Versioning, configure a custom error document, host a webpage on a Windows EC2 instance using IIS, and finally configure a separate S3 bucket to redirect website requests to the EC2 web server.

The practical demonstrates the following operations:

1. Configure an S3 bucket for static website hosting.
2. Enable and demonstrate S3 Versioning.
3. Observe current and previous versions of an object.
4. Download a previous version of an object.
5. Upload the downloaded content again as a new version.
6. Configure and test a custom error document.
7. Use a Windows EC2 instance as a web server through IIS.
8. Configure a separate S3 bucket as a redirect endpoint.
9. Redirect an S3 website request to the Windows EC2 public DNS.

2. Description

This experiment combines Amazon S3 website functionality with a Windows EC2 web server.

The first part of the experiment uses a newly created S3 bucket for static website hosting. The bucket contains website objects including `index.html` and `error.html`. Versioning is enabled so that different versions of `index.html` can be retained and examined.

The experiment begins with the content "Hello World". The file is then modified to "Hello Mister". Because Versioning is enabled, the earlier "Hello World" version remains available while the newer "Hello Mister" version becomes current.

The previous version is subsequently downloaded. Its content is uploaded again, resulting in a new current version containing "Hello World". This demonstrates that S3 creates a new version when content is uploaded again; it does not move the old version.

The second part uses a Windows EC2 instance. The EC2 instance was launched by the root user and IIS is used to serve an HTML page. The webpage displays "Hello from EC2".

A second newly created S3 bucket is then configured for static website redirection. Requests sent to this bucket's website endpoint are redirected to the public DNS address of the Windows EC2 instance.

All S3 buckets used in this experiment were newly created for the experiment.

3. IAM Permissions Added

The existing IAM user was given the following additional S3 permissions for this experiment:

```
s3:GetBucketWebsite
s3:PutBucketWebsite
s3:GetBucketVersioning
s3:PutBucketVersioning
s3:ListBucketVersions
s3:GetObjectVersion
```

These permissions support the website configuration and Versioning operations used in the experiment.

- `s3:GetBucketWebsite` allows the website configuration of a bucket to be read.
- `s3:PutBucketWebsite` allows the website configuration to be created or modified.
- `s3:GetBucketVersioning` allows the Versioning status of a bucket to be read.
- `s3:PutBucketVersioning` allows Versioning to be enabled or changed.
- `s3:ListBucketVersions` allows the available object versions to be listed.
- `s3:GetObjectVersion` allows a particular version of an S3 object to be retrieved.

The IAM user was responsible for the S3-related operations. The Windows EC2 instance itself was launched by the root user.

4. Architecture / Topology

The experiment contains two connected website workflows.

The first workflow uses Amazon S3 as the static website host. The website bucket stores the HTML objects and provides the S3 website endpoint. Versioning operates on the objects stored in this bucket, allowing previous versions to remain available.

The second workflow uses another S3 bucket as a redirect endpoint. This bucket does not contain the final webpage. Instead, its website configuration redirects HTTP requests to the public DNS address of the Windows EC2 instance.

The Windows EC2 instance runs IIS. IIS serves the webpage from the Windows web root:

`C:/inetpub/wwwroot/index.html`

The main components are:

- **S3 static website bucket:** Hosts the S3 website objects.
- **S3 Versioning:** Preserves different versions of `index.html`.
- **Custom error document:** Provides the configured response when requested website content is unavailable.
- **S3 redirect bucket:** Redirects HTTP website requests to the EC2 public DNS.
- **Windows EC2 instance:** Provides the web server used in the second part of the experiment.
- **IIS:** Serves the HTML page hosted on the Windows instance.

The overall request sequence is:

Browser → **S3 Website Endpoint** → **HTTP Redirect**

EC2 Public DNS → **Windows EC2** → **IIS** → `index.html`

The following diagram represents the complete architecture.

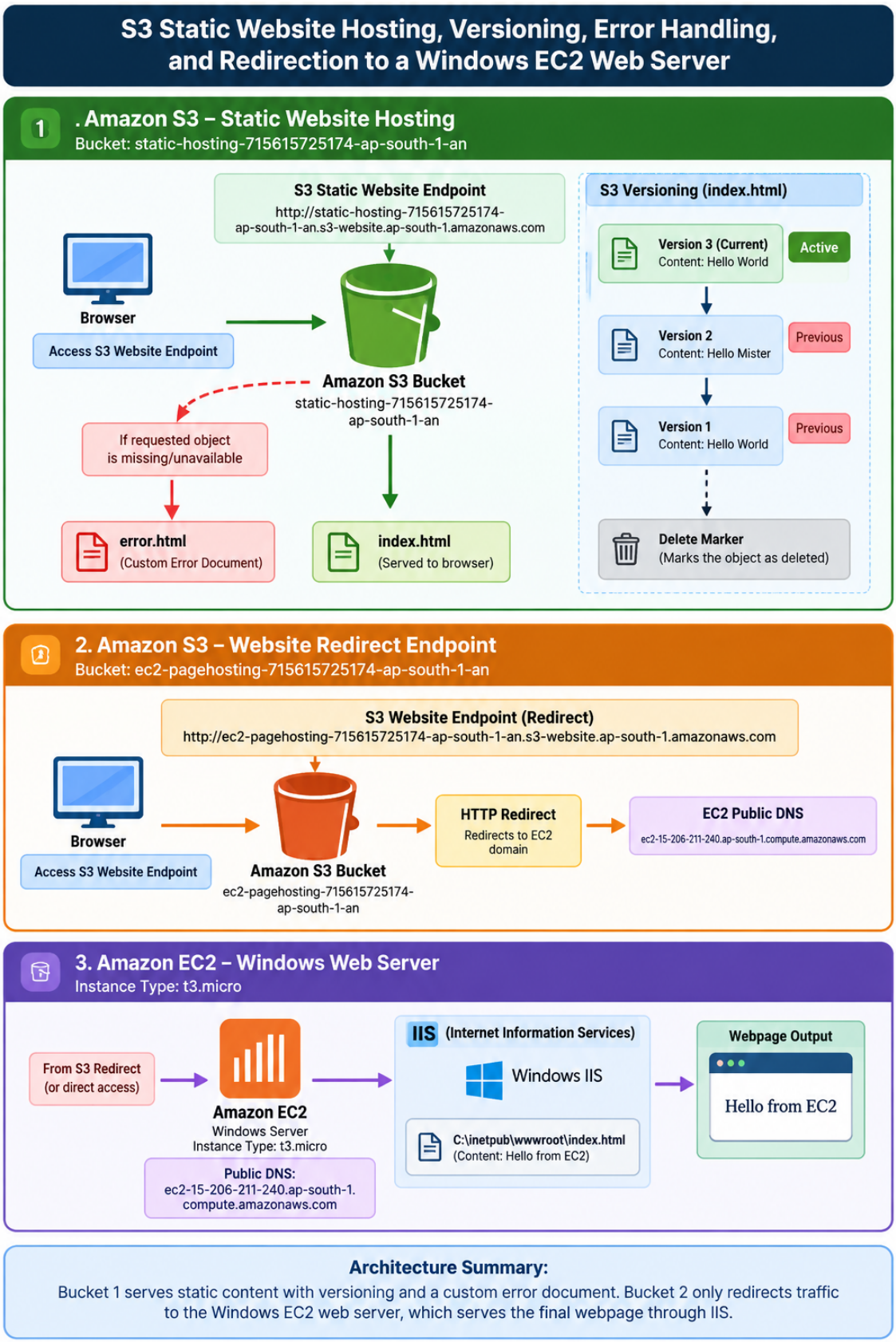


Figure 1: Architecture of the S3 static website, Versioning workflow, Windows EC2 web server, and S3-to-EC2 redirection flow.

5. S3 Static Website Configuration

5.1 Website Bucket

A newly created S3 bucket was used for the static website portion of the experiment.

The bucket used for the website was:

`static-hosting-715615725174-ap-south-1-an`

Static website hosting was configured for the bucket. The website configuration specifies an index document and a custom error document.

The bucket was also configured with the public-access settings required for the website demonstration.

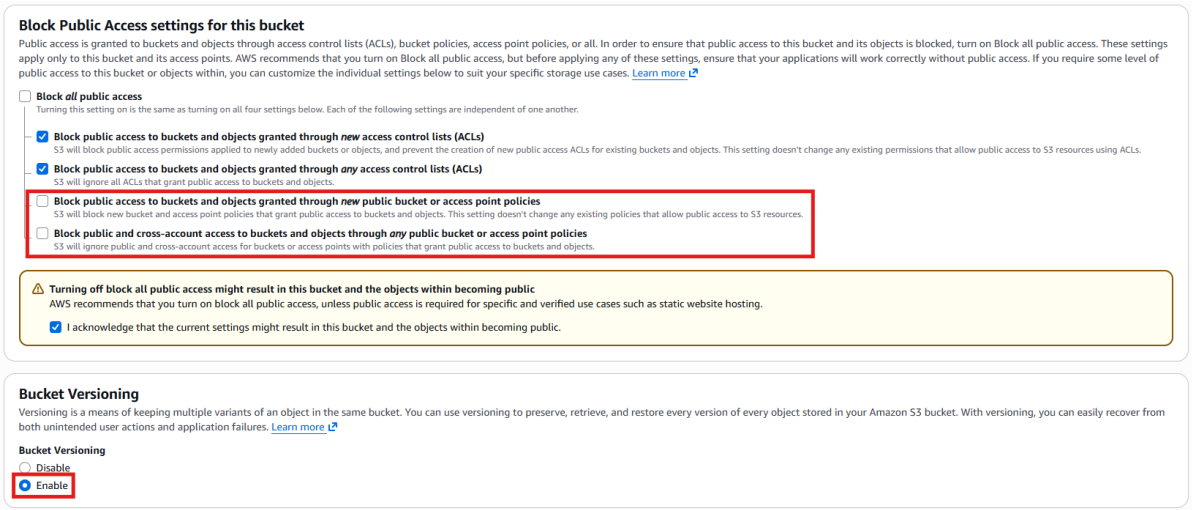


Figure 2: S3 bucket public-access configuration and Versioning settings used for the website experiment.

5.2 Bucket Policy

The website bucket was configured with a bucket policy allowing public `s3:GetObject` access to the required website objects.

This allows a browser to retrieve the objects through the S3 website endpoint.

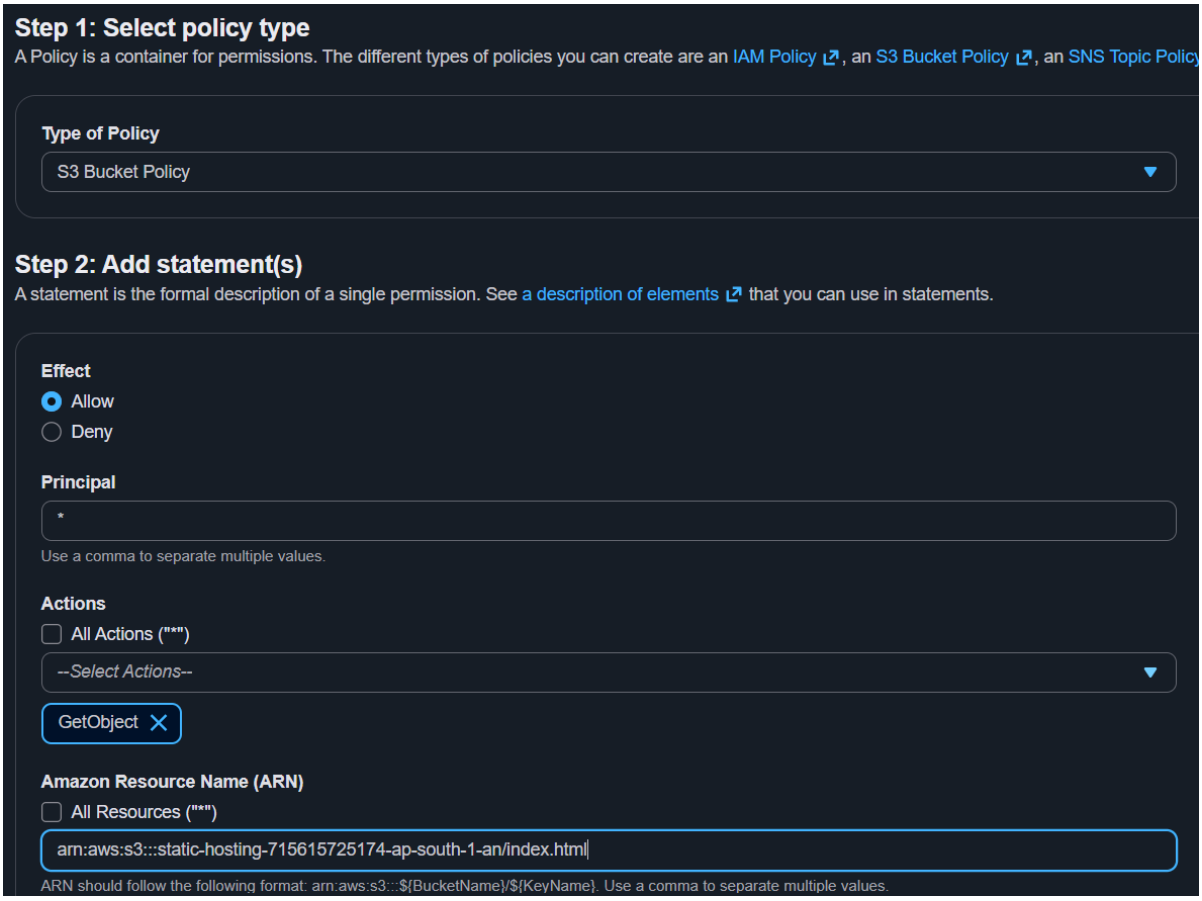


Figure 3: S3 bucket policy configuration used to allow public object retrieval.

5.3 Initial Website Object

The initial `index.html` object contained:

Hello World

This represented the first version of the website content.

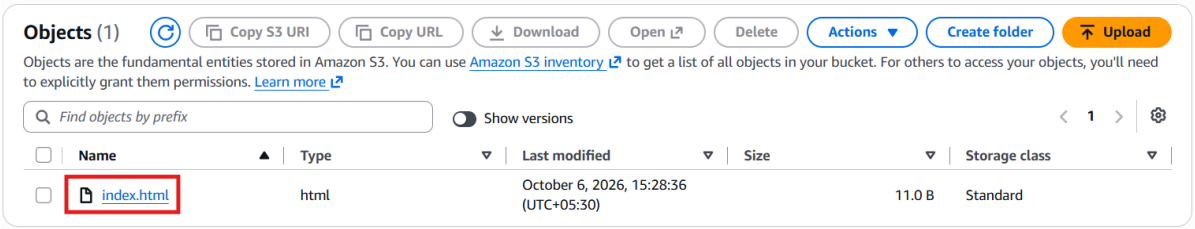


Figure 4: Initial index.html containing “Hello World”.

6. S3 Versioning Demonstration

6.1 Initial and Modified Versions

Versioning was enabled for the S3 bucket.

The initial content was:

Hello World

The content was then changed to:

Hello Mister

The modified file was uploaded again.

Because Versioning was enabled, S3 retained the original object version instead of permanently replacing it.

The resulting state was:

Version state	Content
Current	Hello Mister
Previous	Hello World

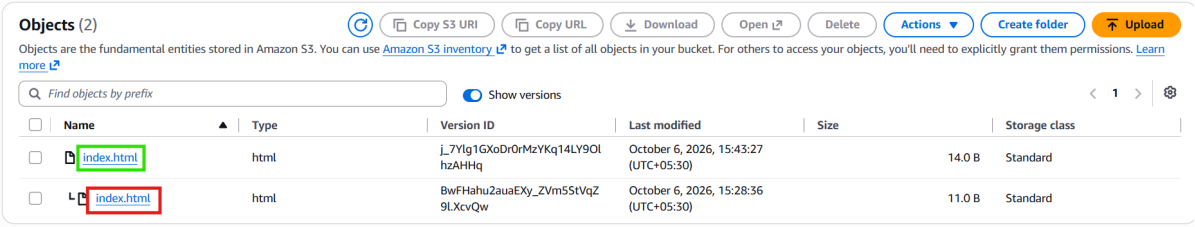


Figure 5: Versioned index.html objects showing the current Hello Mister version and the previous Hello World version.

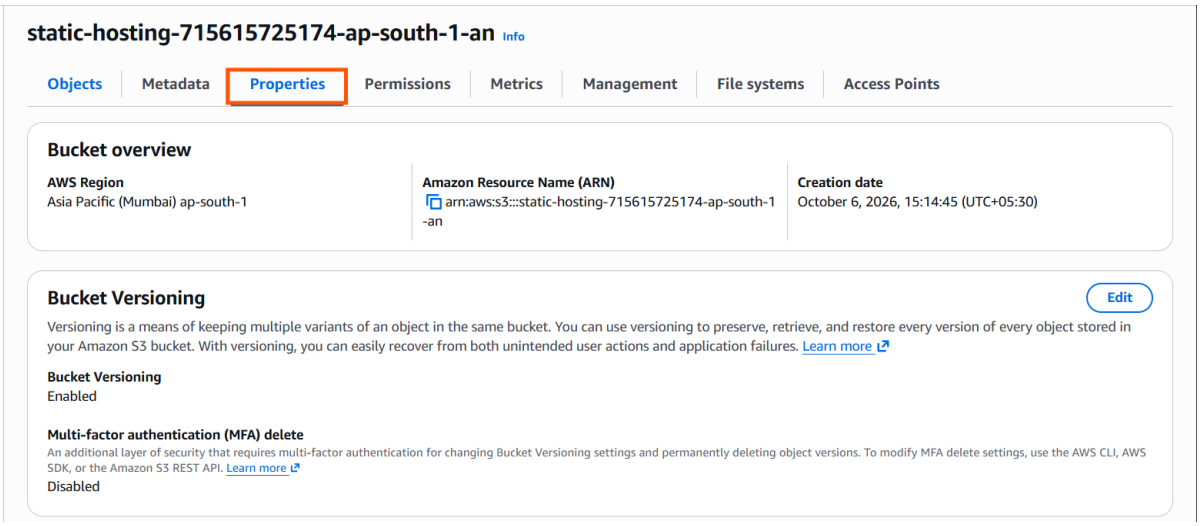


Figure 6: S3 bucket Properties showing that Bucket Versioning is enabled.

6.2 Static Website Showing the Current Version

The S3 website endpoint was accessed after the modification.

The current version displayed:

Hello Mister

This confirmed that the modified version was the active version being served by the website.

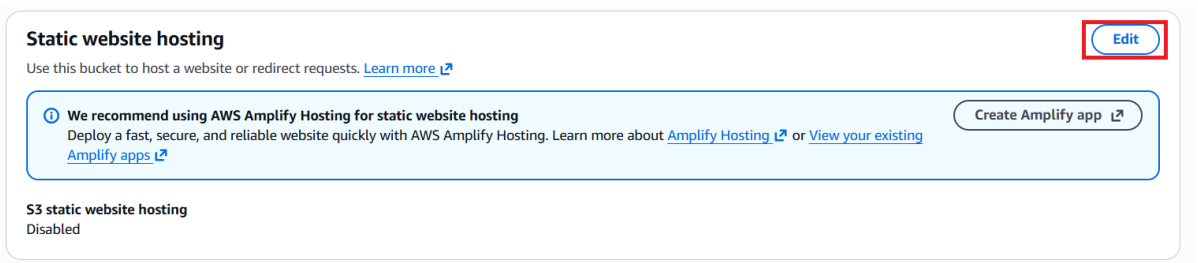


Figure 7: S3 static website hosting configuration showing website hosting enabled.

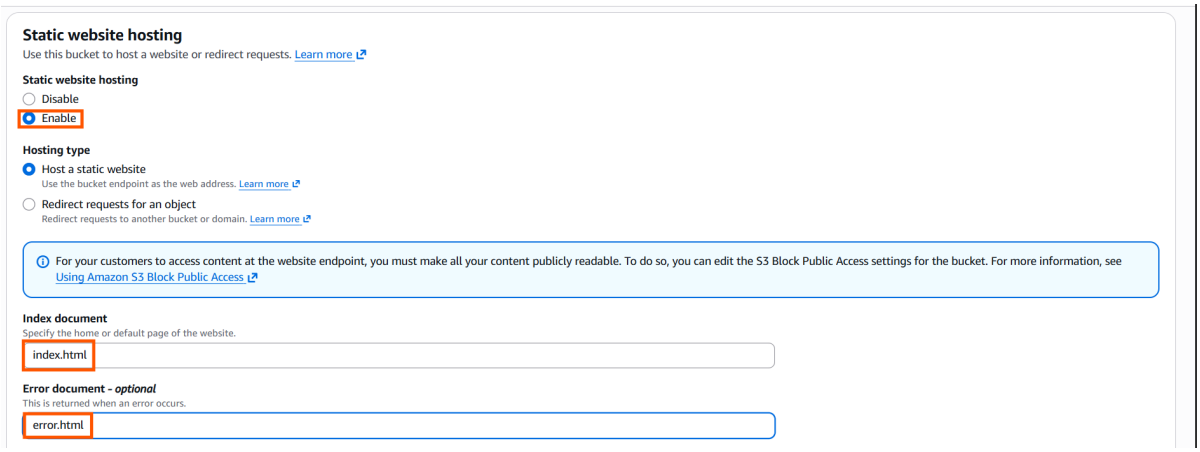


Figure 8: Static website configuration with index.html as the index document and error.html as the error document.

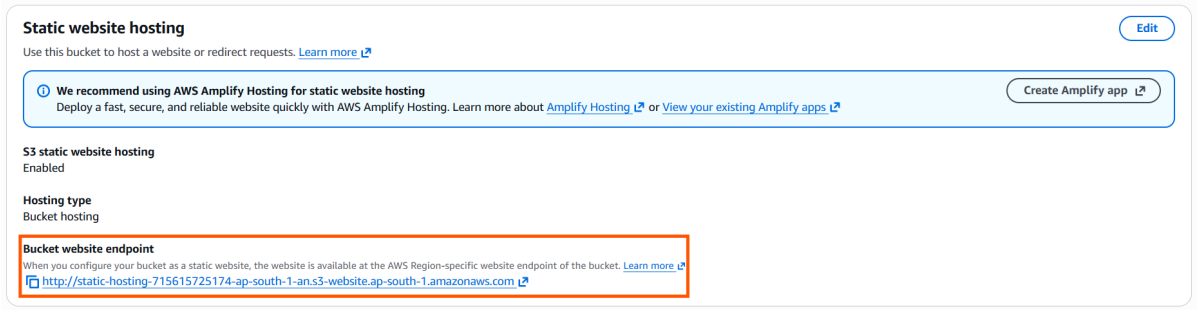


Figure 9: S3 bucket website endpoint generated for the static website.

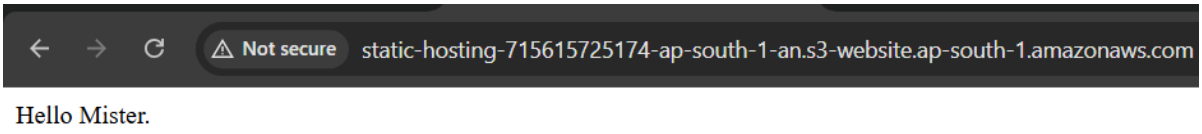


Figure 10: S3 website endpoint displaying the current Hello Mister content.

6.3 Retrieving the Previous Version

The previous “Hello World” version was then selected from the available object versions.

The purpose of this operation was to retrieve the earlier content without deleting or modifying the version stored by S3.

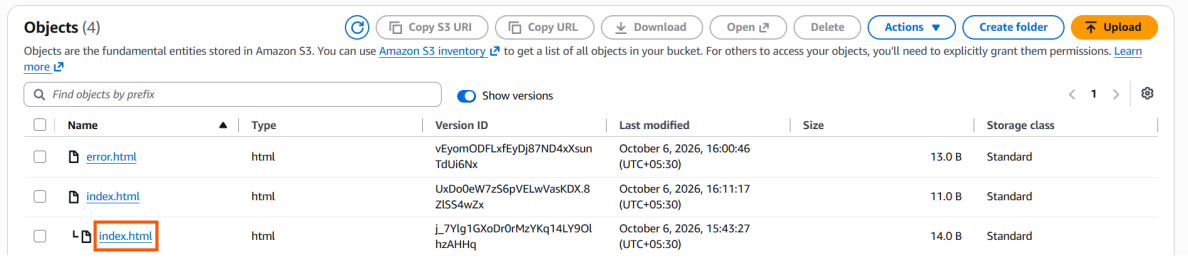


Figure 11: Previous Hello World version of index.html selected from the available object versions.

The previous version was downloaded.

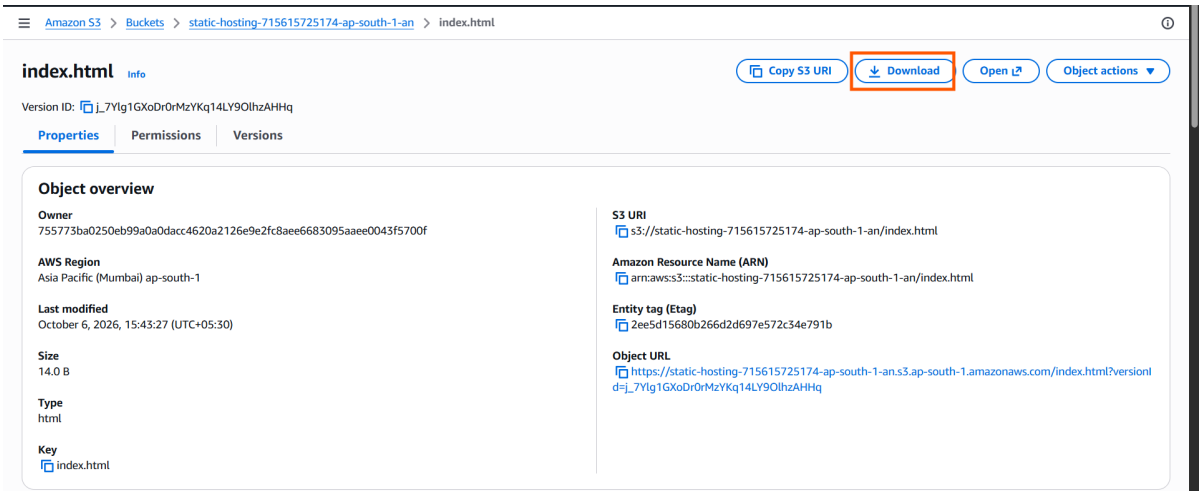


Figure 12: Previous Hello World version being downloaded from S3.

6.4 Uploading the Previous Content Again

The downloaded "Hello World" content was uploaded again as `index.html`.

The resulting sequence was:

Stage	Content
Initial	Hello World
Modified	Hello Mister
Current after modification	Hello Mister
Previous	Hello World
Downloaded previous version	Hello World
Uploaded downloaded content again	Hello World
New current version	Hello World

The important point is that the old "Hello World" version was not moved into the current position.

Instead, the content of that old version was downloaded and uploaded again. S3 therefore created another version containing the same "Hello World" content.

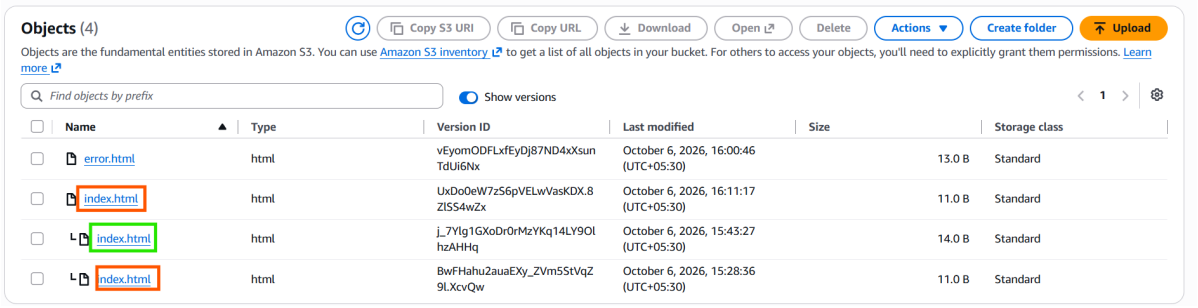


Figure 13: New current version created after uploading the previously downloaded Hello World content.

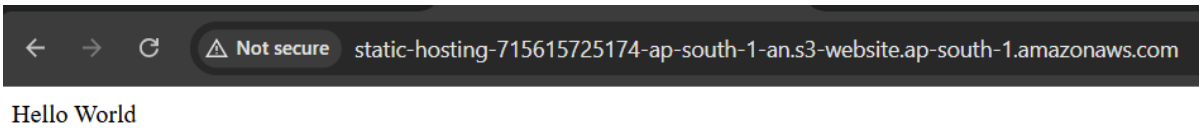


Figure 14: S3 object-version state during the subsequent Versioning workflow.

6.5 Object Deletion and Delete Marker

The versioned `index.html` object was subsequently deleted.

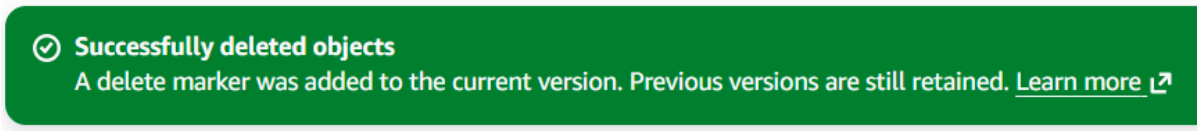


Figure 15: Delete operation performed on index.html while Versioning is enabled.

With Versioning enabled, deleting the object does not immediately remove all preserved versions. A delete marker is added to represent the deletion of the current object state.

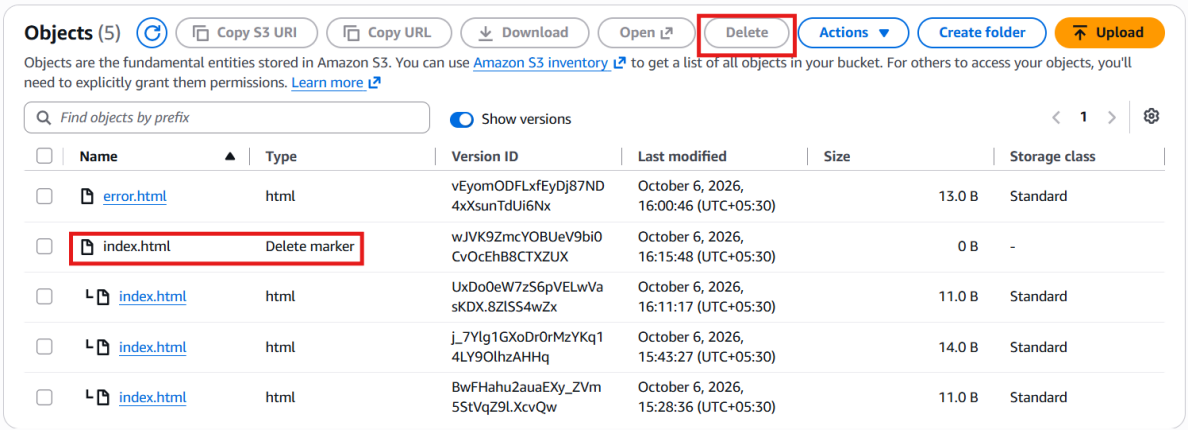


Figure 16: Delete marker shown together with preserved object versions.

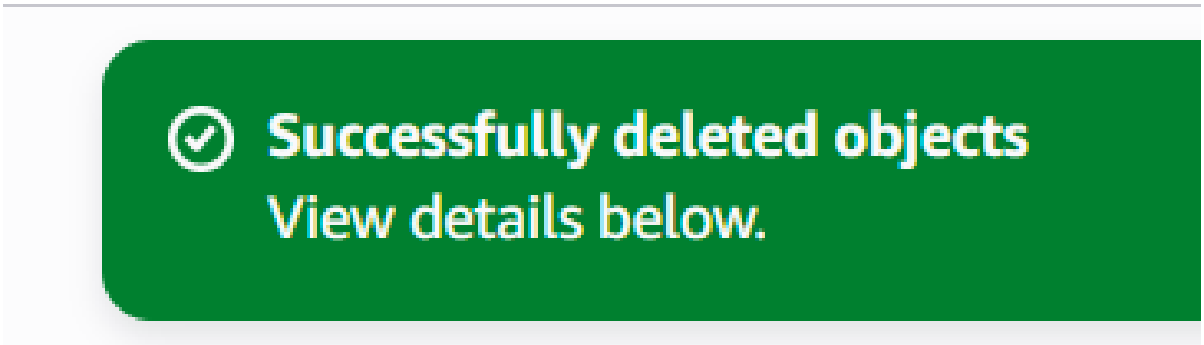


Figure 17: Successful deletion confirmation showing that the delete operation completed.

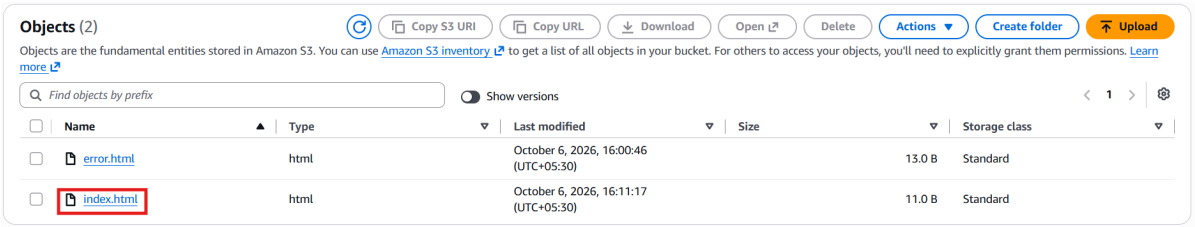


Figure 18: S3 object list after the deletion operation.

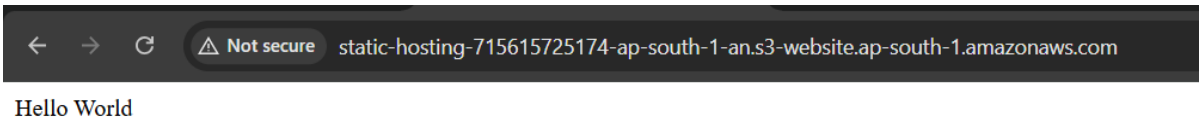


Figure 19: Website output displaying Hello World during the Versioning demonstration.

7. Custom Error Document

The S3 website configuration included `error.html` as the custom error document.

The purpose of this file is to provide a specific webpage when the requested website content cannot be served normally.

The configured error page displayed:

OH NO ERRORRR

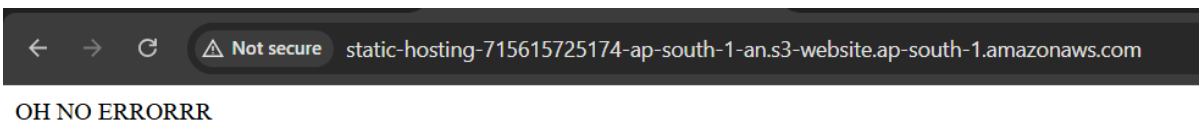


Figure 20: Custom error document displaying “OH NO ERRORRR”.

This demonstrates that the S3 static website configuration can use a separate HTML document for error responses.

8. Windows EC2 + IIS Web Hosting

8.1 Windows EC2 Instance

A Windows EC2 instance was used for the web-server portion of the experiment.

The instance was launched by the root user. The IAM user involved in the S3 work did not launch the EC2 instance.

The EC2 instance was running successfully and its status checks were passing.

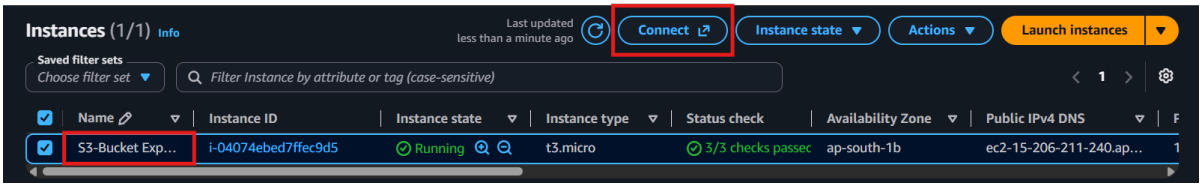


Figure 21: Windows EC2 instance running successfully with passing status checks.

8.2 IIS Web Server

The Windows Server environment contains IIS, which is used to provide the web-server functionality.

The Server Manager dashboard shows IIS available on the server.

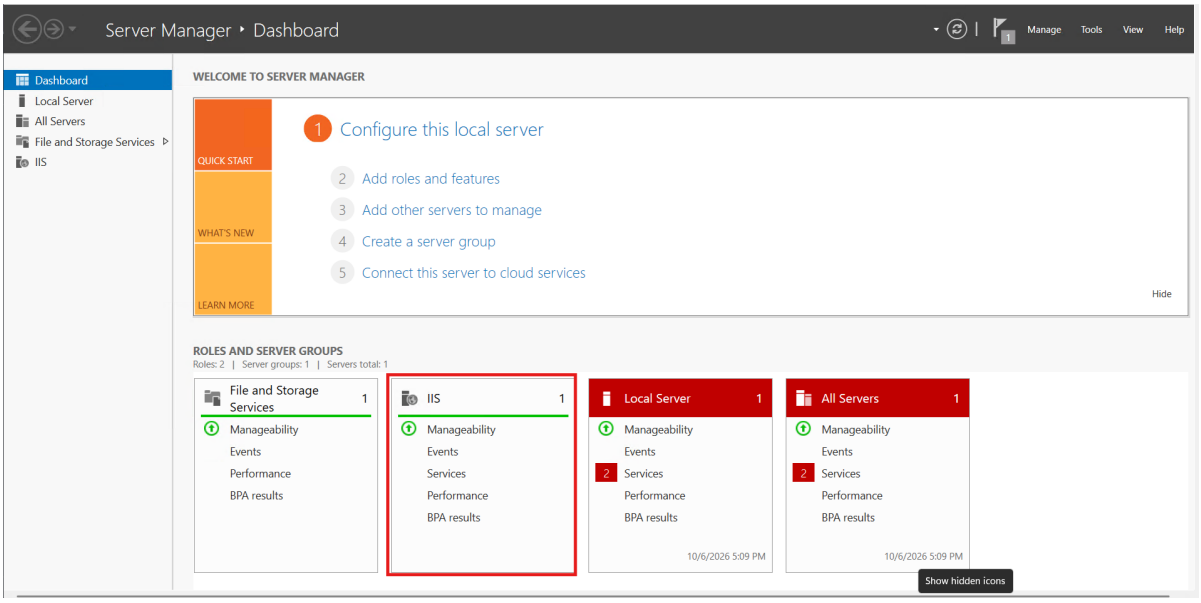


Figure 22: Windows Server Manager showing IIS installed and available.

8.3 HTML Page

The webpage was stored in the IIS web root:

`C:/inetpub/wwwroot/index.html`

The HTML page displayed:

Hello from EC2

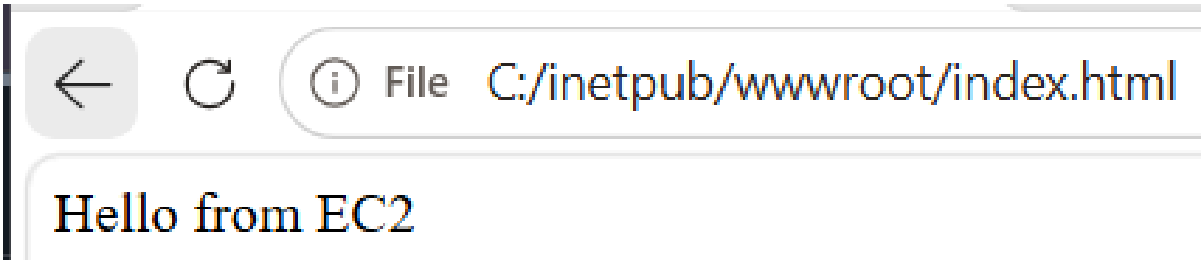


Figure 23: HTML page located in the IIS web root at C:/inetpub/wwwroot/index.html.

The webpage was then accessed through the public DNS address of the EC2 instance.

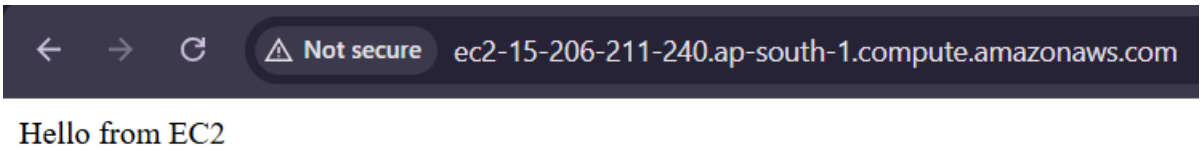


Figure 24: Windows EC2 public DNS displaying the IIS-hosted webpage Hello from EC2.

This confirms that the Windows EC2 instance was independently serving the webpage through IIS.

9. S3 to EC2 Redirect

9.1 Redirect Bucket

A second newly created S3 bucket was used specifically for the redirection portion of the experiment.

The bucket was:

`ec2-pagehosting-715615725174-ap-south-1-an`

This bucket does not provide the final webpage itself. Its purpose is to receive an HTTP website request and redirect the request to the Windows EC2 server.

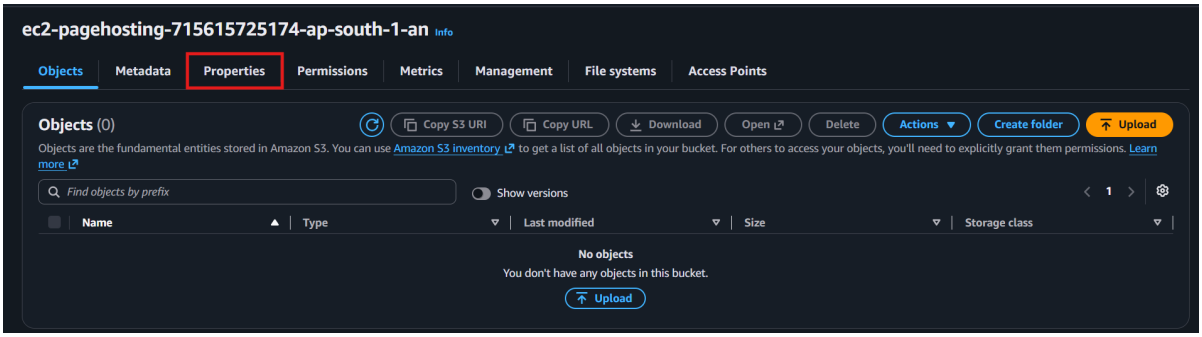


Figure 25: New S3 bucket used for the S3-to-EC2 website redirection experiment.

9.2 Redirect Configuration

Static website hosting was enabled for the redirect bucket.

The hosting type was configured as:

Redirect requests for an object

The target host was the public DNS address of the Windows EC2 instance:

`ec2-15-206-211-240.ap-south-1.compute.amazonaws.com`

The HTTP protocol was selected for the redirect.

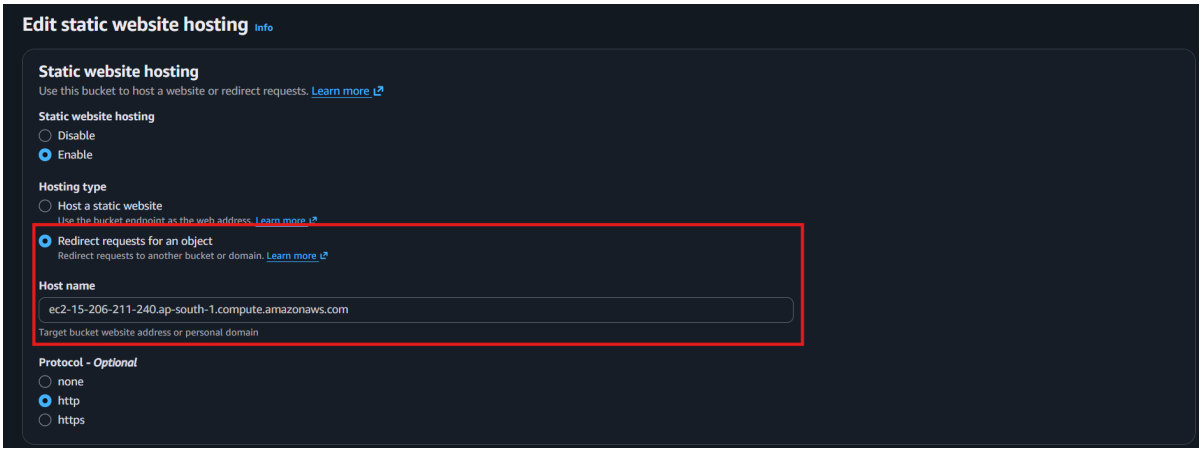


Figure 26: S3 static website configuration using Redirect requests for an object with the EC2 public DNS as the target host.

9.3 Redirect Website Endpoint

S3 generated a website endpoint for the redirect bucket.

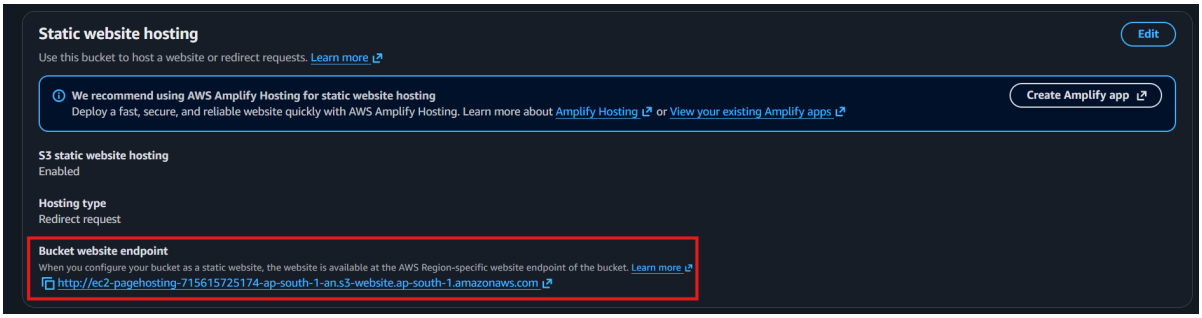


Figure 27: Generated S3 website endpoint for the redirect bucket.

The redirect endpoint acts as the first website-facing endpoint. When a browser accesses it, S3 sends the request toward the configured EC2 destination.

9.4 Final Verification

The S3 redirect endpoint was accessed through a browser.

The request ultimately reached the Windows EC2 instance, where IIS served the HTML page.

The final webpage displayed:

Hello from EC2

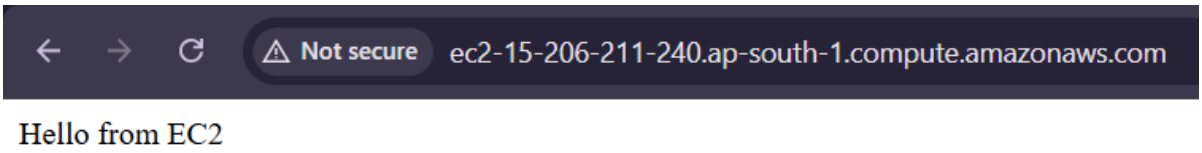


Figure 28: Final browser verification showing the Hello from EC2 page after the S3-to-EC2 redirect.

The final request flow is:

Browser

S3 Website Endpoint → HTTP Redirect

EC2 Public DNS → Windows EC2 → IIS

IIS → `index.html` → Hello from EC2

10. Verification and Results

The experiment demonstrated the following results.

1. A newly created S3 bucket was configured for static website hosting.
2. S3 Versioning was enabled and multiple versions of `index.html` were preserved.
3. The initial "Hello World" content was retained as a previous version after changing the current content to "Hello Mister".
4. The previous "Hello World" version was successfully downloaded.
5. The downloaded content was uploaded again, creating a new current version containing "Hello World".
6. A custom `error.html` document was configured and tested.
7. The Windows EC2 instance launched by the root user successfully hosted a webpage using IIS.
8. The EC2 webpage was accessible through its public DNS address.
9. A second newly created S3 bucket was configured for HTTP redirection.
10. The S3 redirect endpoint successfully directed the request to the Windows EC2 web server.
11. The final browser output displayed "Hello from EC2".

The experiment therefore demonstrates the interaction between Amazon S3 website functionality and a Windows EC2 web server without requiring the S3 redirect bucket to store the final webpage.

11. Conclusion

This practical demonstrated several S3 website features together with a Windows EC2 web server.

The S3 static website configuration showed how HTML objects can be served through an S3 website endpoint. Versioning demonstrated that S3 retains previous object versions when an object is modified.

The Versioning workflow also showed an important distinction between an existing version and a newly uploaded version. The previous "Hello World" content was downloaded and uploaded again, resulting in a new version rather than moving the original version.

The custom error document demonstrated how S3 website hosting can return a separate HTML page when the requested content is unavailable. The second part demonstrated Windows EC2 web hosting through IIS. The EC2 instance was launched by the root user and served an HTML page from the IIS web root. Finally, a separate newly created S3 bucket was configured as a redirect endpoint. The S3 website request was redirected to the public DNS address of the Windows EC2 instance, where IIS returned the final webpage.

The complete practical therefore covered S3 static website hosting, S3 Versioning, previous-version retrieval, custom error handling, Windows IIS hosting, and S3-to-EC2 HTTP redirection.